

The Yenepoya Institute of Arts, Science, Commerce and Management (YIASCM)



DEPARTMENT OF COMPUTER SCIENCE LAB MANUAL

Subject: Introduction to Machine Learning (BCA 401P)

Course Facilitator (s): Ms.Arptha/ Ms.Suzana / Ms.Srijana/
Ms.Jayashrinidhi/Ms Raheema/ Ms
Akshatha/Mr Vikyath

PART- A

1. Write a program to describe the dataset using pandas dataframe commands and encode the same.

Program Code :

```
From sklearn import datasets
```

```
import numpy as np
```

```
import pandas as pd
```

```
from sklearn.preprocessing import LabelEncoder, OneHotEncoder
```

```
#import the dataset
```

```
#from csv file
```

```
data=pd.read_csv(r"C:\Users\jackw\Downloads\iris.csv")
```

```
#from scikit-learn datasets
```

```
iris=datasets.load_iris()
```

```
#convert the dataset into pandas dataframe
```

```
data=pd.DataFrame(data=np.c_[iris['data'],iris['target']],  
columns=iris['feature_names']+['target'])
```

```
#add the target names or label column to the dataframe
```

```
data['target_names']=data['target'].replace(dict(enumerate(iris.target_names)))
```

```
#printing top 5 rows
```

```
print(data.head())
```

```
#display the information iris dataset
```

```
data.info()
```

```
#describe the iris dataset
```

```
print(data.describe())
```

```
#Encoding process
```

```
#Type I-Label Encoding
```

```
#Encoder function
```

```
Label_encoder=LabelEncoder()
```

```
#Labels
```

```
data['target_names']=Label_encoder.fit_transform(data['target_names'])
```

```
print(data['target_names'])
```

```
#Actual labels
```

```

print(Label_encoder.classes_)
#Label count after encoding
print(data['target_names'].value_counts())
#type 2-Label-encoding
#Encoder function
One_hot=OneHotEncoder()
transformed_data=One_hot.fit_transform(data['target_names'].values.reshape(-1,1)).toarray()
print(One_hot.categories_)
#Encode all the labels in the dataset
transformed_data=pd.DataFrame(transformed_data, columns=['setosa','versicolor','virginica'])
print(transformed_data.head())

```

OUTPUT:

```

      sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)
\
0           5.1           3.5           1.4           0.2
1           4.9           3.0           1.4           0.2
2           4.7           3.2           1.3           0.2
3           4.6           3.1           1.5           0.2
4           5.0           3.6           1.4           0.2

```

```

      target target_names
0      0.0      setosa
1      0.0      setosa
2      0.0      setosa
3      0.0      setosa
4      0.0      setosa

```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 150 entries, 0 to 149
```

```
Data columns (total 6 columns):
```

#	Column	Non-Null Count	Dtype
0	sepal length (cm)	150 non-null	float64
1	sepal width (cm)	150 non-null	float64
2	petal length (cm)	150 non-null	float64
3	petal width (cm)	150 non-null	float64
4	target	150 non-null	float64
5	target_names	150 non-null	object

```
dtypes: float64(5), object(1)
```

```
memory usage: 7.2+ KB
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	\
count	150.000000	150.000000	150.000000	
mean	5.843333	3.057333	3.758000	
std	0.828066	0.435866	1.765298	
min	4.300000	2.000000	1.000000	
25%	5.100000	2.800000	1.600000	
50%	5.800000	3.000000	4.350000	
75%	6.400000	3.300000	5.100000	
max	7.900000	4.400000	6.900000	

	petal width (cm)	target
count	150.000000	150.000000
mean	1.199333	1.000000
std	0.762238	0.819232
min	0.100000	0.000000
25%	0.300000	0.000000
50%	1.300000	1.000000
75%	1.800000	2.000000
max	2.500000	2.000000

```
0      0
1      0
2      0
3      0
4      0
```

```
..
```

```
145    2
146    2
147    2
148    2
149    2
```

Name: target_names, Length: 150, dtype: int32

```
['setosa' 'versicolor' 'virginica']
```

```
[array([0, 1, 2])]
```

	setosa	versicolor	virginica
0	1.0	0.0	0.0
1	1.0	0.0	0.0
2	1.0	0.0	0.0
3	1.0	0.0	0.0
4	1.0	0.0	0.0

2. Write a program to normalize the iris dataset using standardization.

Program Code:

```
fromsklearn import datasets
import pandas as pd
fromsklearn.preprocessing import StandardScaler, MinMaxScaler
import warnings

iris = datasets.load_iris()
data = pd.DataFrame(data=iris.data, columns=iris.feature_names)
print(data)
print('Average of Feature')
print(data.mean())
print('\nFeature Variance')
print(data.var())
scaler = StandardScaler()
scaler.fit(data)
iris_scaled_std = scaler.transform(data)
iris_df_std_scaled = pd.DataFrame(data=iris_scaled_std, columns=iris.feature_names)
print('Standardized Feature Average')
print(iris_df_std_scaled.mean())
print('\nStandardized Feature Variance')
print(iris_df_std_scaled.var())
scaler = MinMaxScaler()
scaler.fit(data)
iris_scaled_minmax = scaler.transform(data)
iris_df_minmax_scaled = pd.DataFrame(data=iris_scaled_minmax, columns=iris.feature_names)
print('MinMax Scaled Feature Average')
print(iris_df_minmax_scaled.mean())
print('\nMinMax Scaled Feature Variance')
print(iris_df_minmax_scaled.var())
print('\nMinMax Scaled Min Value')
print(iris_df_minmax_scaled.min())
print('\nMinMax Scaled Max Value')
```

```
print(iris_df_minmax_scaled.max())
```

OUTPUT:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2
..	...	...	...	...
145	6.7	3.0	5.2	2.3
146	6.3	2.5	5.0	1.9
147	6.5	3.0	5.2	2.0
148	6.2	3.4	5.4	2.3
149	5.9	3.0	5.1	1.8

[150 rows x 4 columns]

Average of Feature

sepal length (cm)	5.843333
sepal width (cm)	3.057333
petal length (cm)	3.758000
petal width (cm)	1.199333

dtype: float64

Feature Variance

sepal length (cm)	0.685694
sepal width (cm)	0.189979
petal length (cm)	3.116278
petal width (cm)	0.581006

dtype: float64

Standardized Feature Average

sepal length (cm)	-1.690315e-15
sepal width (cm)	-1.842970e-15
petal length (cm)	-1.698641e-15
petal width (cm)	-1.409243e-15

dtype: float64

Standardized Feature Variance

sepal length (cm)	1.006711
sepal width (cm)	1.006711
petal length (cm)	1.006711
petal width (cm)	1.006711

dtype: float64

MinMax Scaled Feature Average
sepal length (cm) 0.428704
sepal width (cm) 0.440556
petal length (cm) 0.467458
petal width (cm) 0.458056
dtype: float64

MinMax Scaled Feature Variance
sepal length (cm) 0.052908
sepal width (cm) 0.032983
petal length (cm) 0.089522
petal width (cm) 0.100869
dtype: float64

MinMax Scaled Min Value
sepal length (cm) 0.0
sepal width (cm) 0.0
petal length (cm) 0.0
petal width (cm) 0.0
dtype: float64

MinMax Scaled Max Value
sepal length (cm) 1.0
sepal width (cm) 1.0
petal length (cm) 1.0
petal width (cm) 1.0
dtype: float64

3. Write a program to impute the missing values and perform discrete transformation on the iris dataset.

Program Code:

```
from sklearn import datasets
import numpy as np
import pandas as pd
from sklearn.preprocessing import KBinsDiscretizer
from sklearn.impute import SimpleImputer
import warnings

iris = datasets.load_iris()
data = pd.DataFrame(data=iris.data, columns=iris.feature_names)
print(data)

imputer = SimpleImputer(missing_values=np.nan, strategy='mean')
print(imputer)

imputer = imputer.fit(data)
imputed_dataset = imputer.transform(data)
print(imputed_dataset)

iris_quantile_transform = KBinsDiscretizer(n_bins=10, encode='ordinal', strategy='quantile')
quantile_transformed_data = iris_quantile_transform.fit_transform(data)
print(pd.DataFrame(quantile_transformed_data))

iris_uniform_transform = KBinsDiscretizer(n_bins=10, encode='ordinal', strategy='uniform')
uniform_transformed_data = iris_uniform_transform.fit_transform(data)
print(pd.DataFrame(uniform_transformed_data))
```

OUTPUT:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2
..	...	...	...	...
145	6.7	3.0	5.2	2.3
146	6.3	2.5	5.0	1.9
147	6.5	3.0	5.2	2.0
148	6.2	3.4	5.4	2.3
149	5.9	3.0	5.1	1.8

[150 rows x 4 columns]

SimpleImputer()

```
[[5.1 3.5 1.4 0.2]
 [4.9 3.  1.4 0.2]
 [4.7 3.2 1.3 0.2]
```

	0	1	2	3
0	2.0	7.0	1.0	1.0
1	1.0	4.0	1.0	1.0
2	0.0	6.0	0.0	1.0
3	0.0	5.0	2.0	1.0
4	2.0	7.0	1.0	1.0
..	...	...	...	...
145	8.0	4.0	7.0	8.0
146	7.0	1.0	7.0	7.0
147	7.0	4.0	7.0	7.0
148	6.0	7.0	8.0	8.0
149	5.0	4.0	7.0	6.0

[150 rows x 4 columns]

	0	1	2	3
0	2.0	6.0	0.0	0.0
1	1.0	4.0	0.0	0.0
2	1.0	5.0	0.0	0.0
3	0.0	4.0	0.0	0.0
4	1.0	6.0	0.0	0.0
..	...	...	...	...
145	6.0	4.0	7.0	9.0
146	5.0	2.0	6.0	7.0
147	6.0	4.0	7.0	7.0
148	5.0	5.0	7.0	9.0
149	4.0	4.0	6.0	7.0

[150 rows x 4 columns]

4. Write a program to visualize the iris dataset using sepal and petal values.

Program Code:

```
from sklearn import datasets

import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

iris = datasets.load_iris()

data = pd.DataFrame(data=iris.data, columns=iris.feature_names)

print(data)

fig, ax = plt.subplots() # Assign fig here

scatter = ax.scatter(iris.data[:, 0], iris.data[:, 1], c=iris.target)

ax.set(xlabel=iris.feature_names[0], ylabel=iris.feature_names[1])

ax.legend(scatter.legend_elements()[0], iris.target_names, loc="lower right", title="Classes")

plt.show()

fig, ax = plt.subplots()

scatter = ax.scatter(iris.data[:, 0], iris.data[:, 1], c=iris.target)

ax.set(xlabel=iris.feature_names[2], ylabel=iris.feature_names[3])

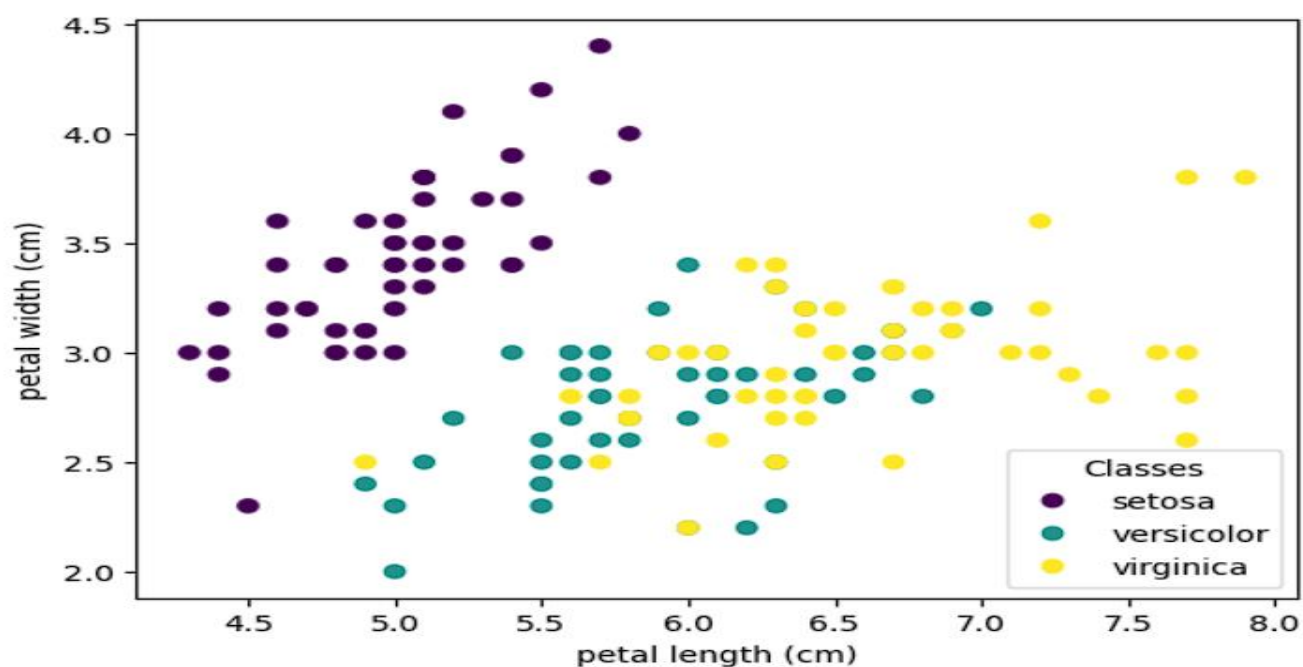
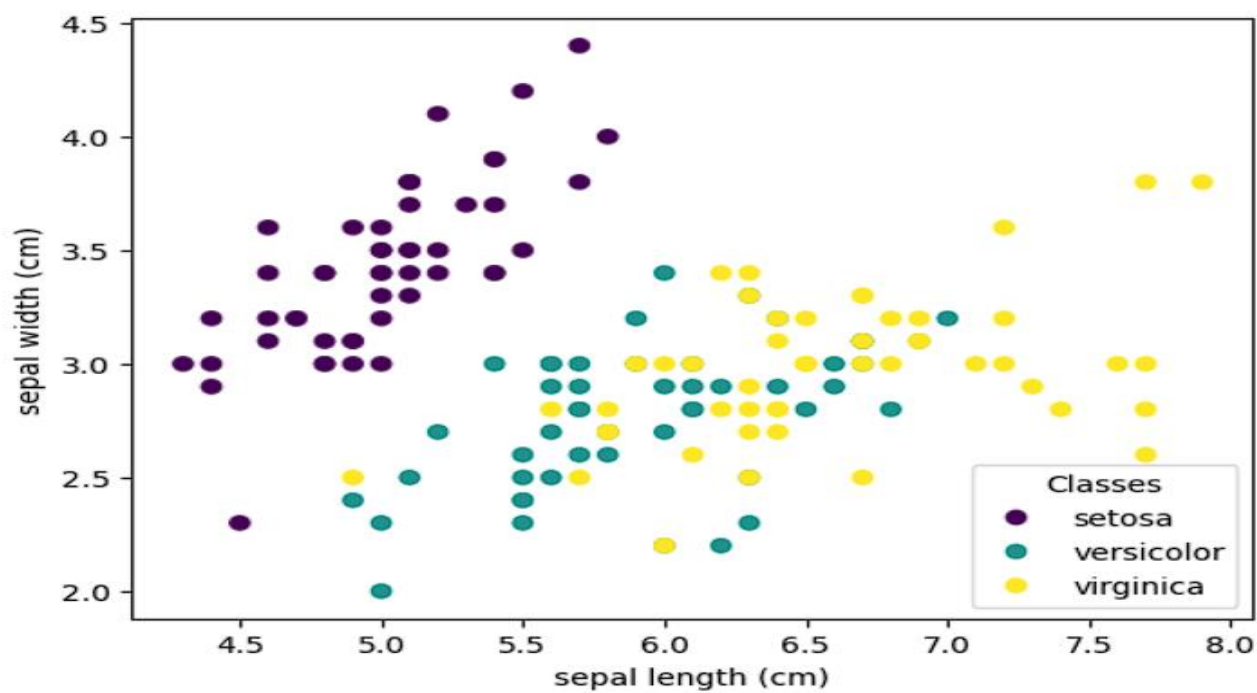
ax.legend(scatter.legend_elements()[0], iris.target_names, loc="lower right", title="Classes")

plt.show()
```

OUTPUT:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2
..	...	...	...	...
145	6.7	3.0	5.2	2.3
146	6.3	2.5	5.0	1.9
147	6.5	3.0	5.2	2.0
148	6.2	3.4	5.4	2.3
149	5.9	3.0	5.1	1.8

[150 rows x 4 columns]



5. Write a program to apply principal component analysis on the iris dataset.

Program Code:

```
from sklearn import datasets, decomposition
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import mpl_toolkits.mplot3d
iris = datasets.load_iris()
data = pd.DataFrame(data=iris.data, columns=iris.feature_names)
X = data
y = iris.target
print(data)
fig = plt.figure(1, figsize=(4, 3))
plt.clf()
ax = fig.add_subplot(111, projection="3d", elev=48, azimuth=134)
ax.set_position([0, 0, 0.95, 1])
plt.cla()
pca = decomposition.PCA(n_components=3)
pca.fit(X)
X = pca.transform(X)
print(X)
for name, label in [("Setosa", 0), ("Versicolour", 1), ("Virginica", 2)]:
    ax.text3D(
        X[y == label, 0].mean(),
        X[y == label, 1].mean() + 1.5,
        X[y == label, 2].mean(),
        name,
        horizontalalignment="center",
        bbox=dict(alpha=0.5, edgecolor="w", facecolor="w"), )
y = np.choose(y, [1, 2, 0]).astype(float)
ax.scatter(X[:, 0], X[:, 1], X[:, 2], c=y, cmap=plt.cm.nipy_spectral, edgecolor="k")
```

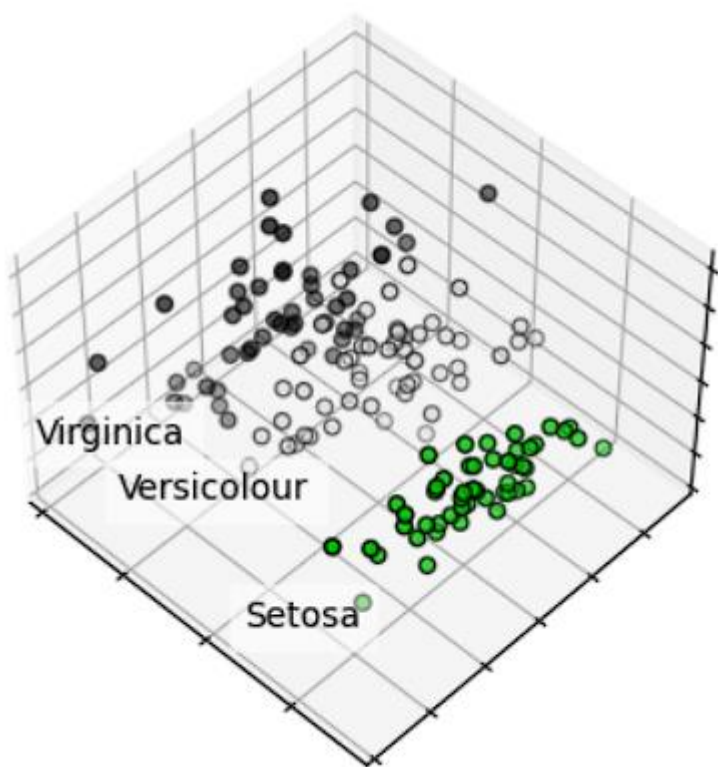
```
ax.xaxis.set_ticklabels([])
ax.yaxis.set_ticklabels([])
ax.zaxis.set_ticklabels([])
plt.show()
```

OUTPUT:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2
..	...	...	...	...
145	6.7	3.0	5.2	2.3
146	6.3	2.5	5.0	1.9
147	6.5	3.0	5.2	2.0
148	6.2	3.4	5.4	2.3
149	5.9	3.0	5.1	1.8

[150 rows x 4 columns]

```
[[-2.68412563  0.31939725 -0.02791483]
 [-2.71414169 -0.17700123 -0.21046427]
 [-2.88899057 -0.14494943  0.01790026]
 [-2.74534286 -0.31829898  0.03155937]]
```



PART- B

6. Write a program to apply linear regression on the iris dataset and find the error of prediction.

```
Ans: from sklearn import datasets, decomposition
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
import matplotlib.pyplot as plt
iris = datasets.load_iris()
X = iris.data
y = iris.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
print("Mean Squared Error:", mse)
```

OUTPUT:

Mean Squared Error: 0.03711379440797688

7. Write a program to apply logistic regression on the iris dataset.


```
Ans: from sklearn import datasets, decomposition
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
iris = datasets.load_iris()
data = pd.DataFrame(data=iris.data, columns=iris.feature_names)
X = data
y = iris.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
logreg = LogisticRegression(solver='lbfgs', penalty='l2', max_iter=10000)
logreg.fit(X_train, y_train)
y_pred = logreg.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
print("Mean Squared Error:", mse)
accuracy = logreg.score(X_test, y_test)
print("Logistic regression Accuracy:", accuracy)
```

OUTPUT:

```
Mean Squared Error: 0.0
Logistic regression Accuracy: 1.0
```

8. Write a program to apply support vector machine classifier on the iris dataset and generate a confusion matrix of the classifier.

Ans: from sklearn import datasets, decomposition

import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.svm import SVC

from sklearn.metrics import confusion_matrix

from sklearn.model_selection import cross_val_score

iris = datasets.load_iris()

X = iris.data

y = iris.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

SVM_classifier = SVC(kernel='linear', random_state=0)

SVM_classifier.fit(X_train, y_train)

accuracy = cross_val_score(estimator=SVM_classifier, X=X_train, y=y_train, cv=10)

print("Accuracy: {:.2f} %".format(accuracy.mean()*100))

y_pred = SVM_classifier.predict(X_test)

SVM_cm = confusion_matrix(y_test, y_pred)

print("The confusion matrix is:")

print(SVM_cm)

OUTPUT:

Accuracy: 95.83 %

The confusion matrix is:

```
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
```

9. Write a program to apply naïve bayes classifier on the iris dataset.

Ans: from sklearn import datasets, decomposition

```
import pandas as pd
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.naive_bayes import GaussianNB
```

```
from sklearn.metrics import accuracy_score
```

```
iris = datasets.load_iris()
```

```
X = iris.data
```

```
y = iris.target
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

```
model = GaussianNB()
```

```
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

```
accuracy = accuracy_score(y_pred, y_test)
```

```
print("Accuracy score is:", accuracy)
```

OUTPUT:

Accuracy score is: 0.9777777777777777

10. Write a program to apply k-means clustering on the iris dataset.

Ans: from sklearn import datasets, decomposition

```
import pandas as pd
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.cluster import KMeans
```

```
import matplotlib.pyplot as plt
```

```
iris = datasets.load_iris()
```

```
X = iris.data
```

```
y = iris.target
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

```
kmeans = KMeans(n_clusters = 3, init = 'k-means++', max_iter = 300, n_init = 10, random_state = 0)
```

```
y_kmeans = kmeans.fit_predict(X)
```

```
plt.scatter(X[y_kmeans == 0, 0], X[y_kmeans == 0, 1], s = 100, c = 'purple', label = 'Iris-setosa')
```

```
plt.scatter(X[y_kmeans == 1, 0], X[y_kmeans == 1, 1], s = 100, c = 'orange', label = 'Iris-versicolour')
```

```
plt.scatter(X[y_kmeans == 2, 0], X[y_kmeans == 2, 1], s = 100, c = 'green', label = 'Iris-virginica')
```

```
plt.scatter(kmeans.cluster_centers_[0, 0], kmeans.cluster_centers_[0, 1], s = 100, c = 'red', label = 'Centroids')
```

```
plt.legend()
```

```
plt.show()
```

OUTPUT:

